

Ex. No.: 11a)
Date: 09/04/25

FIFO PAGE REPLACEMENT

Aim:

To find out the number of page faults that occur using First-in First-out (FIFO) page replacement technique.

Algorithm:

1. Declare the size with respect to page length
 2. Check the need of replacement from the page to memory
 3. Check the need of replacement from old page to new page in memory 4.
- Form a queue to hold all pages
5. Insert the page require memory into the queue
 6. Check for bad replacement and page fault
 7. Get the number of processes to be inserted
 8. Display the values

Program Code:

```
#include <stdio.h>

int main() {
    int frames, pages;
    printf("Enter number of frames: ");
    scanf("%d", &frames);

    printf("Enter number of pages: ");
    scanf("%d", &pages);

    int pageSeq[pages];
    printf("Enter the page reference string:\n");
    for (int i = 0; i < pages; i++)
        scanf("%d", &pageSeq[i]);

    int frame[frames], front = 0, faults = 0;
    for (int i = 0; i < frames; i++) frame[i] = -1;

    printf("\nPage\tFrames\n");
    for (int i = 0; i < pages; i++) {
        int found = 0;
        for (int j = 0; j < frames; j++) {
            if (frame[j] == pageSeq[i]) {
                found = 1;
                break;
            }
        }
    }
}
```

```

    }
}
if (!found) {
    frame[front] = pageSeq[i];
    front = (front + 1) % frames;
    faults++;
}
printf("%d\t", pageSeq[i]);
for (int j = 0; j < frames; j++) {
    if (frame[j] != -1) printf("%d ", frame[j]);
    else printf("- ");
}
printf("\n");
}

printf("\nTotal Page Faults: %d\n", faults);
return 0;
}

```

Sample Output:

[root@localhost student]# python fifo.py

Enter the size of reference string: 20

Enter [1] : 7

Enter [2] : 0

Enter [3] : 1

Enter [4] : 2

Enter [5] : 0

Enter [6] : 3

Enter [7] : 0

Enter [8] : 4

Enter [9] : 2

Enter [10] : 3

Enter [11] : 0

Enter [12] : 3

Enter [13] : 2

Enter [14] : 1

Enter [15] : 2

Enter [16] : 0

Enter [17] : 1

Enter [18] : 7

Enter [19] : 0

Enter [20] : 1

Enter page frame size : 3

7 -> 7 - -

0 -> 7 0 -

1 -> 7 0 1

2 -> 2 0 1
0 -> No Page Fault

3 -> 2 3 1
0 -> 2 3 0
4 -> 4 3 0
2 -> 4 2 0
3 -> 4 2 3
0 -> 0 2 3
3 -> No Page Fault
2 -> No Page Fault
1 -> 0 1 3
2 -> 0 1 2
0 -> No Page Fault
1 -> No Page Fault
7 -> 7 1 2
0 -> 7 0 2
1 -> 7 0 1

Total page faults: 15.
[root@localhost student]#

Result :

Hence the program is compiled and executed successfully.

Ex. No.: 11b)
Date: 09/04/25

LRU

Aim:

To write a c program to implement LRU page replacement algorithm.

Algorithm:

- 1: Start the process
- 2: Declare the size
- 3: Get the number of pages to be inserted
- 4: Get the value
- 5: Declare counter and stack
- 6: Select the least recently used page by counter value
- 7: Stack them according the selection.
- 8: Display the values
- 9: Stop the process

Program Code:

```
#include <stdio.h>

int findLRU(int time[], int n) {
    int min = time[0], pos = 0;
    for (int i = 1; i < n; i++) {
        if (time[i] < min) {
            min = time[i];
            pos = i;
        }
    }
    return pos;
}

int main() {
    int frames, pages;
    printf("Enter number of frames: ");
    scanf("%d", &frames);

    printf("Enter number of pages: ");
    scanf("%d", &pages);

    int pageSeq[pages];
    printf("Enter the page reference string:\n");
    for (int i = 0; i < pages; i++)
        scanf("%d", &pageSeq[i]);

    int frame[frames], time[frames], count = 0, faults = 0;
    for (int i = 0; i < frames; i++) frame[i] = -1;

    printf("\nPage\tFrames\n");
    for (int i = 0; i < pages; i++) {
        int found = 0;
        for (int j = 0; j < frames; j++) {
            if (frame[j] == pageSeq[i]) {
                found = 1;
            }
        }
    }
}
```

```

        time[j] = count++;
        break;
    }
}
if (!found) {
    int pos = -1;
    for (int j = 0; j < frames; j++) {
        if (frame[j] == -1) {
            pos = j;
            break;
        }
    }
    if (pos == -1)
        pos = findLRU(time, frames);
    frame[pos] = pageSeq[i];
    time[pos] = count++;
    faults++;
}

printf("%d\t", pageSeq[i]);
for (int j = 0; j < frames; j++) {
    if (frame[j] != -1) printf("%d ", frame[j]);
    else printf("- ");
}
printf("\n");
}

printf("\nTotal Page Faults: %d\n", faults);
return 0;
}

```

Sample Output :

```
Enter number of frames: 3
Enter number of pages: 6
Enter reference string: 5 7 5 6 7 3
5 -1 -1
5 7 -1
5 7 -1
5 7 6
5 7 6
3 7 6
Total Page Faults = 4
```

Result:

Hence the program is compiled and executed successfully.

Ex. No.: 11c)
Date: 09/04/25

Optimal

Aim:

To write a c program to implement Optimal page replacement algorithm.

ALGORITHM:

1. Start the process
2. Declare the size
3. Get the number of pages to be inserted
4. Get the value
5. Declare counter and stack
6. Select the least frequently used page by counter value
7. Stack them according the selection.
8. Display the values
9. Stop the process

PROGRAM:

```
#include <stdio.h>

int predict(int pages[], int frame[], int n, int index, int frames) {
    int result = -1, farthest = index;
    for (int i = 0; i < frames; i++) {
        int j;
        for (j = index; j < n; j++) {
            if (frame[i] == pages[j]) {
                if (j > farthest) {
                    farthest = j;
                    result = i;
                }
                break;
            }
        }
        if (j == n)
            return i;
    }
    return (result == -1) ? 0 : result;
}
```



```

int main() {
    int frames, pages;
    printf("Enter number of frames: ");
    scanf("%d", &frames);

    printf("Enter number of pages: ");
    scanf("%d", &pages);

    int pageSeq[pages];
    printf("Enter the page reference string:\n");
    for (int i = 0; i < pages; i++)
        scanf("%d", &pageSeq[i]);

    int frame[frames], faults = 0, filled = 0;

    for (int i = 0; i < frames; i++) frame[i] = -1;

    printf("\nPage\tFrames\n");
    for (int i = 0; i < pages; i++) {
        int found = 0;
        for (int j = 0; j < frames; j++) {
            if (frame[j] == pageSeq[i]) {
                found = 1;
                break;
            }
        }

        if (!found) {
            if (filled < frames) {
                frame[filled++] = pageSeq[i];
            } else {
                int pos = predict(pageSeq, frame, pages, i + 1, frames);
                frame[pos] = pageSeq[i];
            }
            faults++;
        }

        printf("%d\t", pageSeq[i]);
        for (int j = 0; j < frames; j++) {
            if (frame[j] != -1) printf("%d ", frame[j]);
            else printf("- ");
        }
        printf("\n");
    }

    printf("\nTotal Page Faults: %d\n", faults);
    return 0;
}

```

Output:

Enter number of frames: 3
Enter number of pages: 12
Enter the page reference string:
7 0 1 2 0 3 0 4 2 3 0 3

Result:

Hence the program is compiled and executed successfully.